CS3101 P1: USQL Interpreter and Query Optimiser

Abstract

This artefact implements two execution modes for a USQL program: an interpreter mode and an optimiser mode. In both modes, the system first parses the USQL program and builds an internal program state containing schemas, loaded relation variables, and query trees. In interpreter mode, a recursive query engine evaluates queries in declaration order, updates state for LET bindings and prints the final query result. In optimiser mode, the final query is inlined and transformed in two stages: a heuristic rewrite phase (simplification and pushdown) followed by cost-based local join reordering. The implementation supports comparison predicates (=, !=, <, <=, >, >=) and Boolean composition with AND/OR. Evaluation with unit tests and end-to-end fixture programs showed correct behaviour on all tested cases.

Introduction

This practical required the implementation of an interpreter and query optimiser for the University of St Andrews Query Language (USQL), a language based on the named relational algebra. A valid USQL program consists of DOMAIN declarations, TYPE declarations, RELATION declarations, LOAD operations, and then one or more queries. The interpreter evaluates a program and prints the result of the final query. The optimiser instead optimises the final query of the program, printing each intermediate step together with the rule applied.

The provided Lark grammar is used to parse USQL source text into a parse tree, which a builder module then walks to produce internal data model objects: domains, types, relations, relational variables loaded from CSV, and a sequence of query expressions. These models form a program state used by both the interpreter and the optimiser. From this point, the two modes diverge. The interpreter runs a query engine, which evaluates each query in order, updating the program state as let statements are encountered, and printing the result of the final query. The optimiser takes the final query expression and inlines all let-bindings into a single expression. It then repeatedly applies one equivalence-preserving rewrite rule at a time in a bottom-up traversal until no further rules apply. These rewrites follow standard relational optimisation heuristics: selections and projections should be pushed as far down the query tree as possible, and selections should be applied before projections, since they tend to reduce the size of intermediate results more. After all structural rules have been applied, a cost-based join reordering pass is performed. It chooses the join order that minimises an estimate of the total number of tuples produced across the entire plan, on the basis that keeping intermediate results small reduces overall computation. Every intermediate step, along with the rule applied, is printed to allow the user to follow the optimisation process. The final optimised query is printed last.

This implementation is tested with a combination of unit tests, which cover individual components in isolation, and end-to-end tests, which run full USQL programs and check both interpreter output and optimizer step traces against expected outputs. The artefact fully satisfies

the coursework specification, supporting all required USQL constructs in both operational modes, with the cost-based join reordering serving as an additional contribution beyond the baseline requirements.

The remainder of this report is organised as follows. The next sections describe the parser, builder, and internal query model, followed by the interpreter and the optimiser. After this, the report explains the cost model and its use in join reordering. Finally, the evaluation section discusses the testing strategy, coverage, robustness checks, and current limitations of the artefact.

Parser, Models, and builder

The Lark parser was provided as part of the coursework. It defines a grammar for USQL and produces a concrete syntax tree (CST) from a source program. One notable modification was making each query form its own named grammar rule (e.g. ‘let_query’, ’select_query', ‘join_query’, etc.) rather than treating all queries as a single flat rule. This makes pattern-matching on specific query types more straightforward when walking the CST in the builder, avoiding needing to check the source code directly to determine the query type. The predicate (theta) rule was extended from equality-only leaves to comparison leaves plus recursive AND/OR composition with explicit precedence (AND binds tighter than OR), so more complex predicates can be parsed into a deterministic predicate tree.

Internal Python classes are used to model the database structure and data from the USQL program. These models largely follow the rules of the grammar: a Domain holds a name and a domain type (int or str), an Attribute holds an attribute name and a reference to a Domain, and a Relation represents a schema (it holds a name and an ordered list of attributes). Additionally, a RelVar pairs a Relation with an actual set of tuples, where each tuple is stored as a dictionary mapping attribute names to values. LOAD operations produce RelVars by reading CSV files and coercing values to the domain referenced by the attributes. LET-bindings also produce RelVars at runtime by assigning the result of a query to a named variable in the program state. Every query operator in both the interpreter and the optimiser receives and returns RelVars.

The query model classes represent the full set of relational algebra operations as a recursive tree structure. Each node in this tree is a data class: SelectQuery, ProjectQuery, UnionQuery, DifferenceQuery, JoinQuery, and RenameQuery are the operator nodes, RelVarQuery is a leaf node referring to a named relvar, LetQuery wraps a query with an assignment target, and EmptyQuery is used by the optimiser to represent an empty relation. Operator nodes hold references to their child query expressions, which may themselves be operator nodes, forming an expression tree that mirrors the algebraic structure of the query. This recursive representation is useful because both execution and optimisation can work on the same structure directly. It also makes local rewrites natural because each rule can match and transform one subtree while leaving the rest unchanged. Predicates for selection queries follow the same approach. The two leaf predicate types are AttrOpAttrPredicate, for comparisons of the form A1

op A2, and AttrOpConstPredicate, for comparisons of the form A op c, where op ∈ {=, !=, <, <=, >, >=}. The recursive AndPredicate and OrPredicate structures allow for the creation of longer predicates with multiple comparisons. AndPredicate may be created directly from the CST or constructed during optimisation to combine a nested selection into a singular one.

The builder walks the CST produced by the parser and constructs a ProgramState object, which acts as the representation of the overall database. The program state holds declared domains, attributes, relation schemas, and relvars. Each is held in a dictionary, mapping their names to the actual object for fast lookup. The ProgramState also holds an ordered list of query objects representing the queries in the program. Building proceeds in declaration order: domains are resolved first, then types, then relations, then loads, and finally queries. This ordering is enforced by the grammar and reflects the dependency structure of a USQL program. The ProgramState produced by the builder is used by both the query engine (interpreter) and the query optimiser.

The builder performs several semantic checks during construction. If a TYPE declaration references a domain name that has not been declared, an error is raised. If a RELATION declaration references an attribute name that has not been declared as a TYPE, an error is raised. If a LOAD operation names a relation that does not exist in the ProgramState, an error is raised. When reading a CSV file, each row is checked to have the correct number of columns for the target relation's schema, and any value corresponding to an integer attribute is coerced from string to integer, raising an error if the conversion fails. These checks ensure that the program state is well-formed before any query evaluation begins.

Interpreter

The query engine is implemented in a single class, QueryEngine, whose run method accepts a ProgramState and iterates through all queries in declaration order, evaluating each in turn. The final evaluated result is returned to the caller and printed. Individual query nodes are dispatched through a recursive '_eval_query' method that pattern matches on the query type using 'isinstance' checks. This recursion aligns with relational algebra: each operator is evaluated from its child expression results, then returns a RelVar result to its parent. Leaf nodes are handled first. A RelVarQuery resolves its name against the ProgramState's relvar dictionary and returns the corresponding RelVar. A LetQuery evaluates its nested query expression and writes the result back into the program state under the target name before returning it. Because the state is mutated in place, later queries in the same program can reference the bound name as though it had been loaded from a file. All queries in the program are evaluated in declaration order, so the final query in the list produces the output that is printed.

Selection is implemented in '_select', which iterates over the tuples of the source RelVar and retains those that satisfy the predicate. Predicate evaluation is recursive: AttrOpAttrPredicate and AttrOpConstPredicate evaluate leaf comparisons using the operator in the node (=, !=, <, <=,

>, >=), while AndPredicate and OrPredicate evaluate their subpredicates then return a result based on simple Boolean logic. An error is raised if a predicate references an attribute name not present in the tuple. The schema of the output RelVar is identical to that of the source.

Projection is implemented in '_project', which constructs a new Relation schema from the specified subset of attribute names, maps each source tuple to the projected key set, and removes duplicate tuples. Deduplication is performed by a '_dedupe_rows' helper that uses a set of seen tuples to filter out repeated rows. This is an intentional choice to make the tuples in a relation a set, and the same duplicate-removal behavior is used in union and join outputs.

The join operator implements a natural join. The shared attribute names (those appearing in both the left and right relation schemas) are identified and used as the join condition. For each pair of tuples from the left and right relations, a combined tuple is produced only when the values of all shared attributes agree. The output schema consists of all attributes from the left relation followed by the right-only attributes, and the result is also deduplicated.

Union and difference both begin by asserting schema compatibility (in _assert_schema_compatible): the two operands must have the same arity with matching attribute names and types at each position. Union concatenates the two tuple lists and deduplicates. Difference uses row signatures (hashable tuples of attribute values in schema order) to build a lookup set for the right operand and retains only left-hand rows whose signature does not appear on the right. Renames in USQL are general renames. They replace all attribute names positionally: a list of new names is applied to the existing attribute list in order, requiring the arity to match exactly, while the underlying domain types are preserved unchanged.

Optimiser

Optimisation is implemented in two phases. This section describes phase one, the heuristic equivalence-rewrite stage. Phase two, cost-based join reordering, is discussed in the next section. The first step in phase one is inlining, which is straightforward: all LET bindings in the final query are substituted for their original queries, so the optimizer operates on a single self-contained expression tree. After inlining, the optimizer repeatedly performs a bottom-up "apply once" pass. In each pass, it recursively descends to the deepest nodes and attempts to rewrite the node. If no applicable rule is found, the parent of the node is checked, and so on. If a successful rewrite is applied, the pass stops immediately, and the rewritten tree becomes the input to the next pass. This process continues until a complete pass finds no applicable rule for any node, terminating phase one. At this point, all trivial structural simplifications have been applied, and projections and selections have been pushed down the expression tree as far as they can go (with selections below projections, when applicable).

Rule application is implemented through structural pattern matching with 'isinstance' checks over the query nodes and their children. Rules are tried in priority order at each node: (1)

trivial simplifications, (2) unary equivalences, (3) selection pushdown, then (4) projection pushdown. This ordering and the rules themselves mirror the equivalence rules in the Query Optimisation lecture (Barwell). It makes behaviour deterministic because the first matching rule in that fixed order is always the one applied, and it starts with the simpler rewrites before the more complicated ones.

The trivial and unary groups are purely structural. They check only the algebraic form of the subtree and do not inspect relation schemas, attribute domains, or tuple data. Examples include identities such as X ∪ X -> X, X − X -> ∅, and interactions with ∅, as well as collapsing nested unary operators (e.g., nested SELECT, PROJECT, or RENAME) into simpler equivalent forms. Because these rewrites depend only on tree shape, they are easy to apply early.

Selection pushdown (through projection, rename, difference, union, and join) contains the first non-trivial rules. For selection pushdown through renaming, the selection predicate attribute names must be translated back to pre-rename names before pushing the SELECT below the rename node. This is handled by constructing a name mapping and rewriting predicate attributes accordingly. When pushing selections past joins, conjunctions are flattened into atomic predicates, then partitioned by referenced attributes: predicates referencing only left-side attributes move to the left input, right-only predicates move to the right input, and cross predicates remain above the join. This preserves semantics while reducing intermediate cardinalities by filtering earlier.

Projection pushdown (through rename, union, and join) is similar but must preserve attributes needed by later operators. Like with selection, projected attribute names must be mapped back to the original names before a projection is pushed below rename. For projection pushdown through join, projection computes per-side keep-sets that include not only the final requested attributes but also attributes needed from the other operand, so the natural join remains valid. Then the inner projections are inserted.

Overall, these rules make selection and projection happen earlier in the evaluation by pushing them down the tree. These operations reduce the cardinality and degree of the relations so applying them before other operations reduces the size of intermediate results. For a concrete optimiser example, see Figure 1 below, which shows the output from running the optimiser on a test file. You can see a series of selection pushdowns as well as the final optimised query on the final line.

```
Initial inlined query: σ[lcr=rcr](σ[rcr=15](σ[lms=2]((ρ[lmc, lmt, lms, lcr](module) ⋈ ρ[rmc, rmt, rms, rcr](module)))))
Selection pushdown through join: σ[lcr=rcr](σ[rcr=15]((σ[lms=2](ρ[lmc, lmt, lms, lcr](module)) ⋈ ρ[rmc, rmt, rms, rcr](module))))
Selection pushdown through rename: σ[lcr=rcr](σ[rcr=15]((ρ[lmc, lmt, lms, lcr](σ[ms=2](module)) ⋈ ρ[rmc, rmt, rms, rcr](module))))
Selection pushdown through join: σ[lcr=rcr]((ρ[lmc, lmt, lms, lcr](σ[ms=2](module)) ⋈ σ[rcr=15](ρ[rmc, rmt, rms, rcr](module))))
Selection pushdown through rename: σ[lcr=rcr]((ρ[lmc, lmt, lms, lcr](σ[ms=2](module)) ⋈ ρ[rmc, rmt, rms, rcr](σ[cr=15](module))))
σ[lcr=rcr]((ρ[lmc, lmt, lms, lcr](σ[ms=2](module)) ⋈ ρ[rmc, rmt, rms, rcr](σ[cr=15](module))))
```

Figure 1: This screenshot shows the output produced when the command 'sh run.sh --o tests/fixtures/pushdown_join.usql' is run from the project directory. It shows the optimisation of the final query in the specified USQL file. The let-bound relvars are first inlined into the query, and then a sequence of selection pushdown rewrites is applied. The final optimised query is printed on the final line.

Cost-based Optimiser

Phase two optimises join order using estimated cost. Join order can drastically change intermediate cardinalities even though the final result is the same. Each join operation yields an estimated result size (number of tuples), and those estimations are accumulated to form a total plan cost for the given join chain ordering. This total cost can then be used to select the most efficient ordering of the joins. For example, in a three-way join, joining two large weakly-related relations first can create a huge temporary relation, while joining a highly selective pair first can keep the intermediate small. The cost model is therefore used to choose the parenthesization with the lowest estimated total work. Reordering is applied only to joins because join order can change intermediate cardinalities while preserving equivalence, whereas operators such as selection, projection, rename, union, and difference are handled via deterministic algebraic rewrites in phase one rather than cost-based permutation search.

This is done as a separate pass from phase one because it is not just another local rewrite rule. Phase one works by pattern matching a node and immediately replacing it with a single preferred equivalent form. It continues until no rule applies to the tree. Cost-based join ordering evaluates alternative equivalent join trees and selects the lowest-cost option. There is no single pattern to match on and replace, meaning if done in phase one, it could repeat forever and continually match on already-optimised join chains.

Operationally, phase two is a single bottom-up traversal of the query tree. When a JoinQuery node is visited, if both operands of the join are not joins themselves (e.g. $r \bowtie s$), nothing is done because there is no alternative parenthesisation or reordering that would have any impact. If exactly one operand is itself a join, the current node exposes three join operands (e.g. $r \bowtie (s \bowtie u)$). If both operands are joins, it exposes four (e.g. $(r \bowtie t) \bowtie (s \bowtie u)$). Those 3- or 4-operand frontiers are then enumerated into alternative binary join trees, each candidate is costed, and the cheapest one is chosen (with deterministic tie-breaking by generation order). This is valid because natural joins are reorderable as equivalent join trees, so changing parenthesization preserves semantics. The method is intentionally local: if a child subtree contains an even deeper join chain, that subtree is treated as atomic here because it has already been optimized earlier in the bottom-up traversal. As a result, the algorithm does not exhaust every possible ordering for arbitrarily long join chains, because exhaustive global search grows combinatorially and can make optimisation itself too expensive relative to the expected gain.

Cost estimation is computed recursively in a style very similar to query evaluation in the interpreter: visit children first, then compute the parent's output statistics from child statistics. For each node, the estimator returns four values: estimated output rows $n(r)$, output attribute list, distinct-value estimates $V(A, r)$ per attribute, and cumulative cost. Here cumulative cost means the total estimated rows produced by all intermediate results in the plan. Therefore, join orderings that keep intermediate results small will have smaller cumulative costs and be preferred over orderings that have large intermediate results. Base relations use exact statistics

from the relevant Relvar model in the ProgramState. Selection, projection, rename, union, difference, and join use the formulas found in the Query Optimisation lecture (Barwell). For example, the estimated size of a projection operation $\pi A(r)$ is $V(A, r)$ (the number of distinct values for the attribute A in r).

The result of phase two is therefore a cost-informed choice among equivalent join trees. Combined with phase one, this gives a two-stage optimizer: first reduce and clean the expression shape, then choose better join orderings where order affects execution cost.

Evaluation

The evaluation uses a layered testing strategy: unit tests for parser and builder behaviour, query-model construction, query-engine operator semantics, heuristic optimiser rewrites, and cost-based join reordering, alongside end-to-end tests that execute full USQL programs in both interpreter and optimiser modes. The suite was run using 'pytest/unittest', with 94 tests passing, providing both targeted validation of individual components and full pipeline verification.

Coverage includes valid and invalid parsing, builder semantic errors, and correctness of all relational operators (selection, projection, join, union, difference, and rename). Correctness is further demonstrated through targeted operator tests. For example, projection tests verify that duplicate rows are removed after attribute reduction, ensuring set semantics. Join tests check that tuples are combined only when shared attribute values agree, and that the resulting schema preserves left attributes followed by right-only attributes in the expected order. Optimisation behaviour is validated through both structural and cost-based cases. For example, pushdown tests confirm that selections above joins are rewritten so single-side predicates are applied to the appropriate input before the join. Cost-based tests verify that, for nested joins, the optimiser selects a join grouping with lower estimated cumulative cost than alternative orderings. Robustness checks include handling of unknown references, malformed CSV rows, integer coercion failures, rename arity mismatches, empty-query propagation, and schema compatibility enforcement for set operators.

There are some limitations to this project. The implementation supports all constructs in the provided USQL grammar, but the language itself could be extended. Predicate support now includes inequalities and Boolean AND/OR, but the current Boolean grammar does not include unary NOT. The cost model also uses simplifying assumptions (e.g., near-uniform value distribution, predicate independence, and simple fallback formulas for less-specified comparison cases), which can reduce estimate accuracy on skewed data. On the optimisation side, join reordering is local rather than global. An exhaustive global search could produce better plans for some deep join chains, but with much higher optimisation cost.

Conclusion

This project delivers a complete USQL pipeline with clear stage boundaries: the parser and builder construct a validated program state, the interpreter evaluates queries recursively in declaration order, and the optimiser transforms the final inlined query via heuristic rewrites followed by a separate cost-based join-order pass. This structure worked well because each stage has a distinct responsibility while sharing one query-tree representation, so execution and optimisation remain semantically aligned.

Beyond the baseline requirements, the implementation significantly extends USQL predicate support to comparison operators (=, !=, <, <=, >, >=) and Boolean composition (AND/OR), and adds cost-based local reordering of join chains to reduce estimated intermediate result sizes. Overall, the test results provide strong practical evidence of correctness for both required functionality and these extensions. Unit tests validate individual components, while end-to-end tests confirm that full programs produce the expected interpreter results and optimizer traces.

References

Barwell, Adam D. “Query Manipulation and Optimisation.” CS3101.